

Experiment 'p_6_9_arrays_equal_seq' Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_6_9_arrays_equal_seq

Natural language statement: null

Method signature: p_6_9_arrays_equal_seq(first: seq<int>, second: seq<int>) returns (are_equal: bool)

Ensures

- are_equal == (|first| == |second| && forall i :: 0 <= i < |first| ==> first[i] == second[i])

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_6_9_arrays_equal_seq(first: seq<int>, second: seq<int>) returns (are_equal: bool)`

The method should respect the following contract:

`ensures are_equal == (|first| == |second| && forall i :: 0 <= i < |first| ==> first[i] ==
↪ second[i])`

Produce and show only the Dafny body of this method, including the curly braces that
↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
↪ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnOQ6DoOr2slgtkWJ9rV8wyUMz7Hf

Model: gpt-5.5-2026-04-23

Created at: 1780664314

Finish reason: stop

Usage: 310 tokens in, and 3617 tokens out

```
{  
  if |first| != |second| {  
    are_equal := false;  
  } else {  
    are_equal := true;  
    var i := 0;  
    while i < |first|  
      invariant |first| == |second|  
      invariant 0 <= i <= |first|  
      invariant are_equal ==> forall j :: 0 <= j < i ==> first[j] == second[j]  
      invariant !are_equal ==> exists j :: 0 <= j < i && first[j] != second[j]  
      decreases |first| - i  
    {  
      if are_equal {  
        if first[i] != second[i] {  
          are_equal := false;  
          assert 0 <= i < i + 1 && first[i] != second[i];  
          assert exists j :: 0 <= j < i + 1 && first[j] != second[j];  
        } else {  
          assert first[i] == second[i];  
        }  
      }  
    }  
  }  
}
```

```

    forall j | 0 <= j < i + 1
      ensures first[j] == second[j]
    {
      if j < i {
        assert first[j] == second[j];
      } else {
        assert j == i;
        assert first[j] == second[j];
      }
    }
  }
} else {
  ghost var k :| 0 <= k < i && first[k] != second[k];
  assert 0 <= k < i + 1 && first[k] != second[k];
  assert exists j :: 0 <= j < i + 1 && first[j] != second[j];
}
i := i + 1;
}

assert i == |first|;
if are_equal {
  assert forall j :: 0 <= j < |first| ==> first[j] == second[j];
} else {
  assert exists j :: 0 <= j < |first| && first[j] != second[j];
  ghost var k :| 0 <= k < |first| && first[k] != second[k];
  assert !(forall j :: 0 <= j < |first| ==> first[j] == second[j]);
}
}
}

```

Final Program

```

method p_6_9_arrays_equal_seq(first: seq<int>, second: seq<int>) returns (are_equal: bool
  ↪ )
  ensures are_equal == (|first| == |second| && forall i :: 0 <= i < |first| ==>
    ↪ first[i] == second[i])
{
  if |first| != |second| {
    are_equal := false;
  } else {
    are_equal := true;
    var i := 0;
    while i < |first|
      invariant |first| == |second|
      invariant 0 <= i <= |first|
      invariant are_equal ==> forall j :: 0 <= j < i ==> first[j] == second[j]
      invariant !are_equal ==> exists j :: 0 <= j < i && first[j] != second[j]
      decreases |first| - i
    {
      if are_equal {

```

```

    if first[i] != second[i] {
      are_equal := false;
      assert 0 <= i < i + 1 && first[i] != second[i];
      assert exists j :: 0 <= j < i + 1 && first[j] != second[j];
    } else {
      assert first[i] == second[i];
      forall j | 0 <= j < i + 1
        ensures first[j] == second[j]
      {
        if j < i {
          assert first[j] == second[j];
        } else {
          assert j == i;
          assert first[j] == second[j];
        }
      }
    }
  }
} else {
  ghost var k :| 0 <= k < i && first[k] != second[k];
  assert 0 <= k < i + 1 && first[k] != second[k];
  assert exists j :: 0 <= j < i + 1 && first[j] != second[j];
}
i := i + 1;
}

assert i == |first|;
if are_equal {
  assert forall j :: 0 <= j < |first| ==> first[j] == second[j];
} else {
  assert exists j :: 0 <= j < |first| && first[j] != second[j];
  ghost var k :| 0 <= k < |first| && first[k] != second[k];
  assert !(forall j :: 0 <= j < |first| ==> first[j] == second[j]);
}
}
}

```

Total Token Usage

Input tokens: 310

Output tokens: 3617

Reasoning tokens: 3072

Sum of 'total tokens': 3927

Experiment Timings

Overall Experiment started at 1780664313425, ended at 1780664392608, lasting 79183ms (79.18 seconds)

Iteration #1 started at 1780664313425, ended at 1780664392608, lasting 79183ms (79.18 seconds)